Altu Bozyaka (s2572629)

Computer Graphics: Rendering (CGR) 2026 · Coursework Checkpoint 1

Module 1: A Whitted-style and a distributed ray tracer

As part of Module 1,

- **I built:** A renderer that reads scenes written in the pbrt-v4 format and renders them with either the **Whitted-style ray tracer** or the **distributed ray tracer**. All 13 required features in Module 1 work (Table 1). Every render produces a PNG to look at, a PFM with the raw HDR values, and a log with errors and timings.
- **How I show it:** To highlight every feature, I annotated the renders (the yellow numbered markers like 1 point at the exact spot), created small test scenes, and calculated numbers to check the actual results against theoretical formulas.
- **Creative scene:** I took my creative photo on a pool table at Teviot Row House (Fig. 7a), and recrated in 3D with a virtual camera matched to the photo and rendered the same scene with both tracers (Figures 8 and 15).

How the code is organised: The renderer is written in C++17 using only the standard library, about 4900 lines in total. It is developed and tested on macOS Apple M5 Pro (15 cores). The general architecture works in this order: a scene file is parsed into a Scene (scene/), rendered in parallel 16×16 tiles by the integrator it decides, and tone mapped and saved as PNG, PFM and .log (core/), with shapes in geometry/ and the camera, materials, lights and integrators in render/. Overall, the WSRT and DRT tracers share everything except the per-ray shading step.

2 Whitted-style ray tracer (WSRT)

A Whitted tracer shoots one ray per sample, shades the nearest hit with Blinn-Phong and point lights, and then follows perfect mirror and refraction rays recursively. In my code this is WhittedIntegrator::trace(ray, depth, w), where w is the product of all the reflection and transmission factors picked up so far. Figure 1 is the scene I built to show all of it: one object per feature, I chose a checkerboard floor to make distortions easy to read, and two point lights so that every object casts two shadows that are easy to observe.

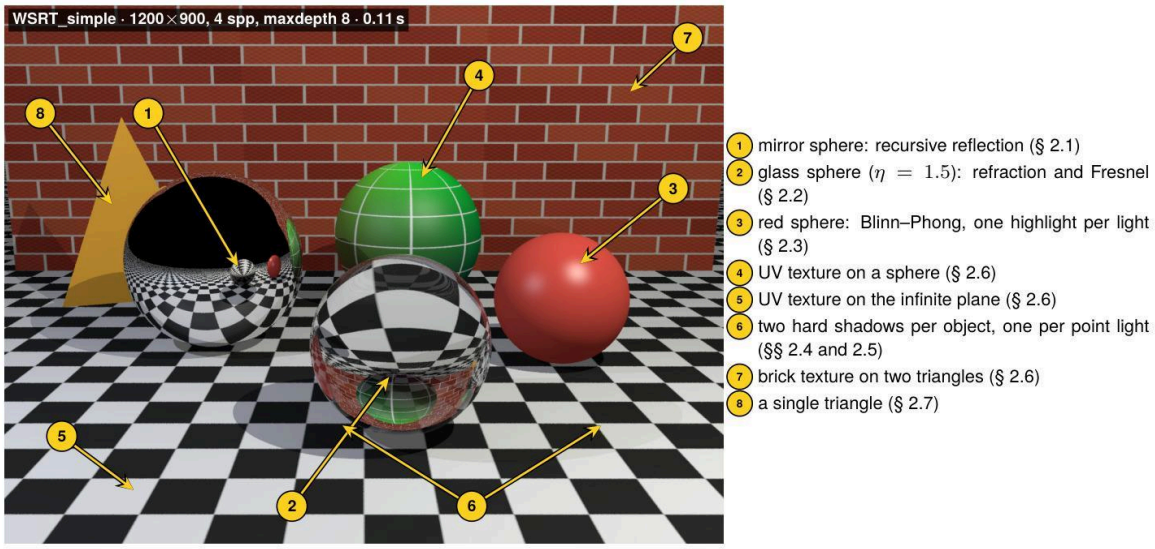


Figure 1: WSRT_simple: every WSRT feature in one image. The key light at front right is much stronger than the bluish fill light at front left so each object's darker shadow belongs to the key light.

2.1 Specular reflection

What I did and why: The reflected direction is the standard $\mathbf{r} = -\boldsymbol{\omega}_o + 2(\boldsymbol{\omega}_o \cdot \mathbf{n})\mathbf{n}$. A "mirror" material returns K_r times whatever that ray sees, and any Phong material can add a K_r term too (0.75 for the steel grater in the creative scene, 0.05 for the clear coat on the pool balls). I stopped the recursion at integer maxdepth (default = 5), *or* when $w < 0.001$. I added this second rule because a ray that small can't change an 8-bit pixel's brightness, and with glass, where every hit splits into two rays, the ray tree otherwise doubles at every level and becomes computationally very heavy.

How I tested it: I re-rendered WSRT_simple with maxdepth 1, 2, 3 (Figure 2). Counting the surfaces along a path predicts when the glass sphere should appear inside the mirror: mirror + two glass surfaces = depth 3, and that is exactly when it appears. Going from depth 3 to 8 changes the image by only 1.3×10^{-4} on average, all inside the glass, so depth 8 is already high and costs just 18 % more rays than depth 1.

Challenges: The first preview of my creative scene (water glass, bottle and crystal cap at maxdepth 12) took 161 s at only 400×300, because the ray tree exploded inside the glass. The $w < 10^{-3}$ cut-off made it 31 times faster (5.2 s) with no visible difference.

Observation: The bricks sparkle at the mirror's rim, where one pixel covers a lot of reflected wall.

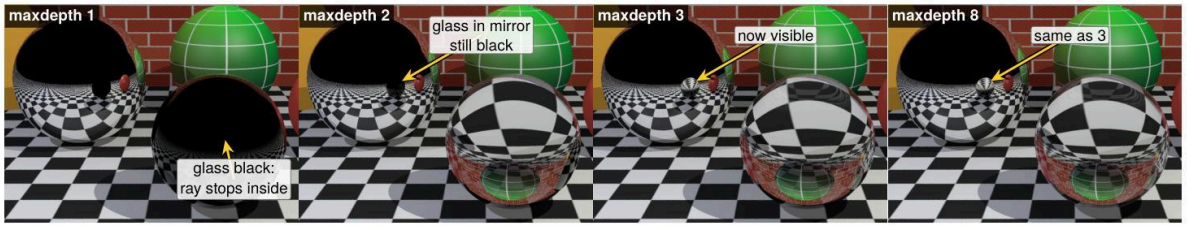


Figure 2: Recursive reflection, the mirror sphere **1** and glass sphere **2** at four depths. At depth 1 the refracted ray stops inside the glass, so we see the glass as black. The glass sphere seen in the mirror needs one more level than the glass itself, so it only appears at depth 3. The black top half of the mirror is rays escaping into the black background.

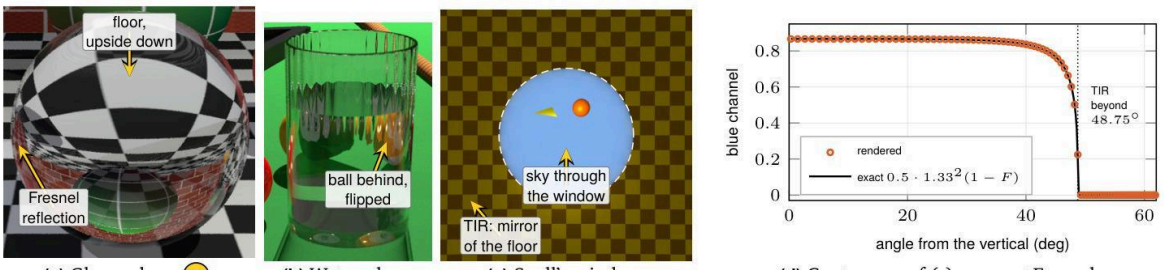
2.2 Specular refraction: Snell's law, Fresnel and total internal reflection

What I did and why: refractDir applies Snell's law, $\eta_1 \sin\theta_i = \eta_2 \sin\theta_t$, in vector form. Whether a ray is entering or leaving the glass comes from the sign of $\omega_o \cdot n$ against the outward normal, so nothing has to remember that a ray is "inside". If $\sin\theta_t$ would be above 1 there is no refracted ray: that is total internal reflection, and only the reflection is gonna be traced. The split between the two rays uses the exact Fresnel equations rather than Schlick's approximation, because they are exact near the critical angle, which is where Schlick approximation performs the worst. The refracted ray is weighted by $(1 - F)K_t/\eta'^2$, where the $1/\eta'^2$ accounts for radiance changing when light crosses into a denser medium.

How I tested it: A glass sphere can never show TIR (a ray leaves it at the same angle it entered), so I built a separate test, Snell's window (Figure 3c,d): a camera under water looking straight up. The sky is only visible inside a 48.75° cone, and outside it the surface acts as a perfect mirror. Only the sky contains blue, so the blue channel measures exactly the light transmitted through the surface, $0.5 \cdot 1.33^2(1 - F)$. The render follows the exact curve (RMS error 0.0015), and the edge of the window lands on the predicted pixel.

Challenges: The first version of this test disagreed by 0.14 at the edge of the window. The "water surface" is really a huge sphere, and its tiny 0.065° tilt matters there because $1 - F$ drops vertically at the critical angle so including it in the expected curve fixed the mismatch.

Observation: The water glass in the creative scene behaves like the real life in my photo: the ball behind it is magnified and flipped left to right (Figure 3b).



(a) Glass sphere (2). **(b)** Water glass. **(c)** Snell's window. **(d)** Centre row of (c) vs exact Fresnel.

Figure 3: Refraction. (a) The glass sphere acts as a ball lens, so the view through it is upside down, and Fresnel reflection brightens its rim. (b) In the creative scene, the water cylinder magnifies the ball behind it and mirrors it. (c, d) The TIR test: sky light only gets through inside the critical angle but outside it the surface reflects the pool floor completely, and the measured transmission follows the Fresnel curve right down to the sudden drop at 48.75°.

2.3 Blinn-Phong shading

What I did and why: At each hit on a Phong material the WSRT adds an ambient term and, for each light the point can see ($V_j = 1$), a diffuse and a specular term:

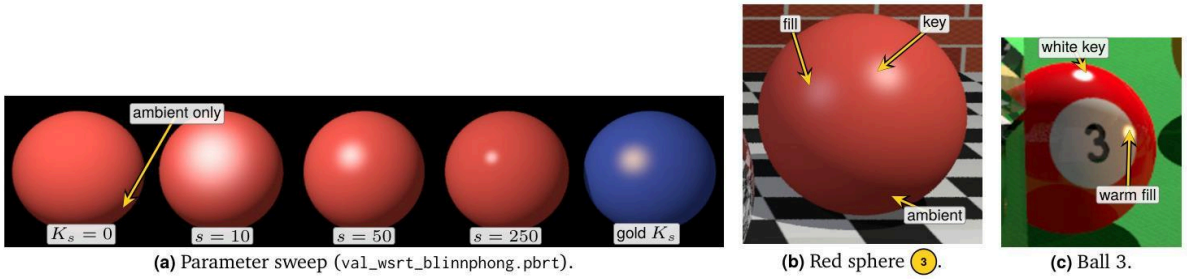
$$L = K_a K_d I_a + \sum_j V_j \frac{I_j}{d_j^2}\left[K_d \max(0, \mathbf{n}\cdot\mathbf{l}_j) + K_s \max(0, \mathbf{n}\cdot\mathbf{h}_j)^s\right], \qquad \mathbf{h}_j = \frac{\mathbf{l}_j + \mathbf{v}}{\|\mathbf{l}_j + \mathbf{v}\|}.$$

K_d (a texture), K_s, the shininess s, K_a and K_r are all set per material in the scene file (Material "blinnphong" ...), and I_a by Integrator "rgb ambient". I basically kept the classic, unnormalised formula from the lectures.

How I tested it: The sweep in Figure 4a changes one thing at a time. With $K_s = 0$ only the diffuse shading and ambient term remain. Raising s from 10 to 250 shrinks the highlight while its peak stays at $K_s I/d^2$, and a gold K_s on a blue sphere gives a gold highlight. The diffuse term is checked exactly by the point-light test below.

Problems: In the creative scene the pillar and walls looked flat and too bright to me, because the ambient term was lighting surfaces that should be dark. So when I lowered their K_a , I managed to fixx it. The log also warns when $K_d + K_s > 1$, which Blinn-Phong allows but the DRT wouldn't allow it..

Observation: A shinier surface reflects less light in total (same peak, narrower lobe). That isn't physical, and it is one reason the DRT uses a normalised lobe instead.



(a) Parameter sweep (`val_wsrt_blinnphong.pbrt`). **(b)** Red sphere (3). **(c)** Ball 3.

Figure 4: Blinn-Phong. (a) The highlight narrows as the shininess s rises and takes the colour of K_s the no-light side shows only the ambient term. (b, c) Each light makes its own highlight in its own colour.

2.4 Point light sources

What I did and why: LightSource "point" "point3 from" [..] "rgb I" [..] gives a position and a radiant intensity, and the light arriving at a point is I/d^2. I used intensity rather than power because pbrt does, so the same numbers give the same brightness in both renderers. In the creative scene a white key light hangs above the far end of the table, with a warm 3000 K fill light from the right.

How I tested it: I placed a flat plane under one light, with one sample through each pixel centre (Figure 5a, b). For all 30 000 pixels the script computes the exact $K_d I \cos\theta/d^2$, and the worst relative error is 5.9×10^{-8}, which is just due to float rounding. I compared the renderer against pbrt-v4 with test 5 (a floor under a point light) and it agrees within 3.4 %.

Problems. None, it passed first time.

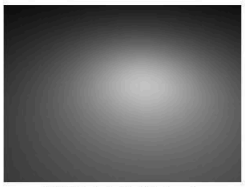
2.5 Shadows

What I did and why: Each light gets one shadow ray, using an “any hit” test that stops at the first blocker, since it only matters whether something is in the way. To avoid shadow acne (where rounding puts the hit point slightly under the surface and the shadow ray then hits its own surface) the renderer makes it so that every new ray starts a tiny distance off the surface along the geometric normal, $\varepsilon = 10^{-7}(1 + \max|p_i|)$. I made it relative because rounding errors grow with the size of the coordinates.

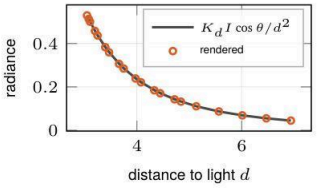
How I tested it: WSRT_simple shows no shadow acne anywhere, even on the far checkerboard where rays hit the floor at a shallow angle and acne would show there first, and each object has a dark key light shadow and a lighter fill shadow (Figure 5c).

Problems: My creative photo has coloured, see-through shadows under the glass and the bottle, but in a Whitted tracer glass blocks shadow rays completely. I added an option, "bool transparentshadows", which dims a shadow ray by $K_t(1 - F)$ at each glass surface instead (Figure 5d), plus I added pbrt’s alpha cut-outs for the holes in the grater instead of modelling every hole as geometry(Figure 5e). An analytic check with a glass slab above a floor gives 0.4608 against the expected 0.4608. Both options are off by default, and WSRT_simple renders identically without them.

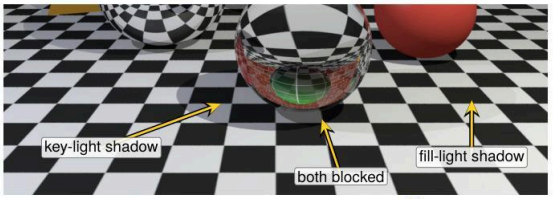
Observation: The bright, focused spot inside the glass’s shadow in my real-life creative photo is a caustic. A Whitted tracer can’t produce it, because shadow rays only travel in straight lines to the light.



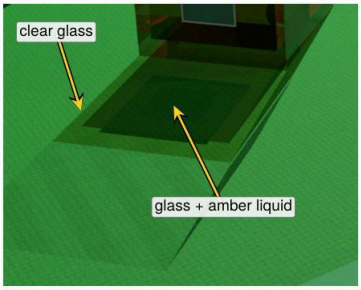
(a) Point-light test.



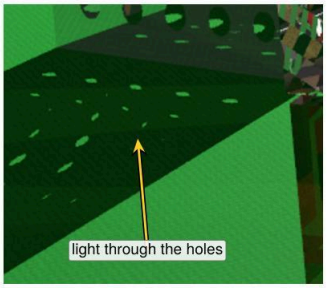
(b) Centre row vs formula.



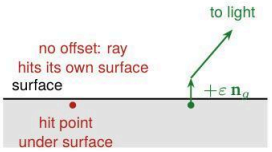
(c) Two hard shadows per object 6.



(d) Filtered bottle shadow.



(e) Grater shadow (alpha).



(f) Why rays start off the surface.

Figure 5: Point lights and shadows. (a, b) On a flat floor the light falls off as $\cos\theta/d^2$ and the render follows the formula pixel for pixel. (c) The darker shadow belongs to the stronger key light, where both lights are blocked and only the ambient light is left. (d, e) The two creative scene options: a tinted, see-through glass shadow, and spots of light through the grater’s holes. (f) The ray origin offset that prevents shadow acne.

2.6 Textures

What I did and why: Textures are PPM (or PFM) files read by my own loader into memory and converted from sRGB to linear once on loading, because lighting has to be computed in linear units (a stored 0.5 is really 0.214, so skipping this makes textures look washed out). Texture lookups blend the four nearest pixels, and I followed pbrt’s tiling (uscale/vscale) and flipped v so scenes look the same in both renderers. Each shape works out its own texture coordinates (u,v): spheres use their angles like longitude and latitude, planes use the hit’s position on the plane, and triangles blend the UVs stored at their corners.

How I tested it: Fig. 6 shows each texture file next to where it exists in the scene: grid lines getting closer towards the sphere’s pole, checker tiles staying the same size across the infinite plane, and bricks repeating 3 - 2 - 3 - 2 texture pattern, all look really good overall.

Problems: The pool balls’ numbers first came out mirrored, because pbrt’s φ runs right to left when seen from outside. I fixed it in the scene with Scale -1 1 1 on the balls rather than changing the texture code, so the renderer stays prbt-compatible. My ball texture was also only 148×75 pixels per ball and looked blurry at first, so the textures were regenerated at 768×384 in the same colours.

2.7 Geometry: planes, spheres and triangles

What I did and why: All shapes share one interface (intersect, bounds, area, sample), so lights and integrators treat them the same way. Spheres use pbrt's quadratic form, triangles use Möller-Trumbore and the infinite plane is my own extension, Shape "plane", since pbrt doesn't have one.

How I tested it: Comparing outlines with pbrt-v4 checks intersections and the camera together: only 0.057 % of pixels differ (marked red), all on edges (Fig. 6, bottom right).

Problems:d Without a BVH (hopefully will do in later modules), a 9 216-triangle mesh was far too slow, so as a solution every object got one bounding box so if a ray misses the box skips all its triangles.

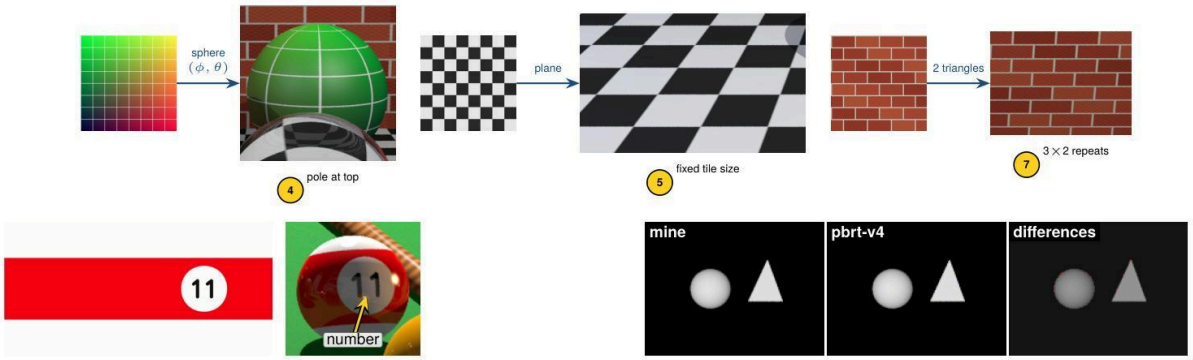
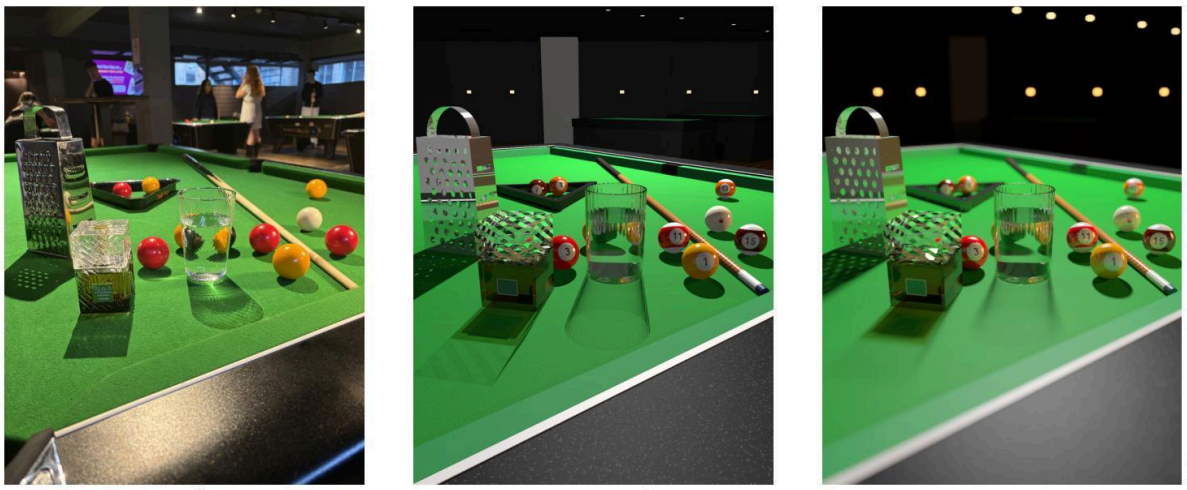


Figure 6: Textures and geometry. Top: each texture file and where it ends up in WSRT_simple. Bottom left: one of the generated pool ball textures and the ball it wraps. Bottom right: the same sphere and triangle rendered by my renderer and by pbrt-v4. As a result, only 11 of 19 200 pixels differed (red), all on edges.

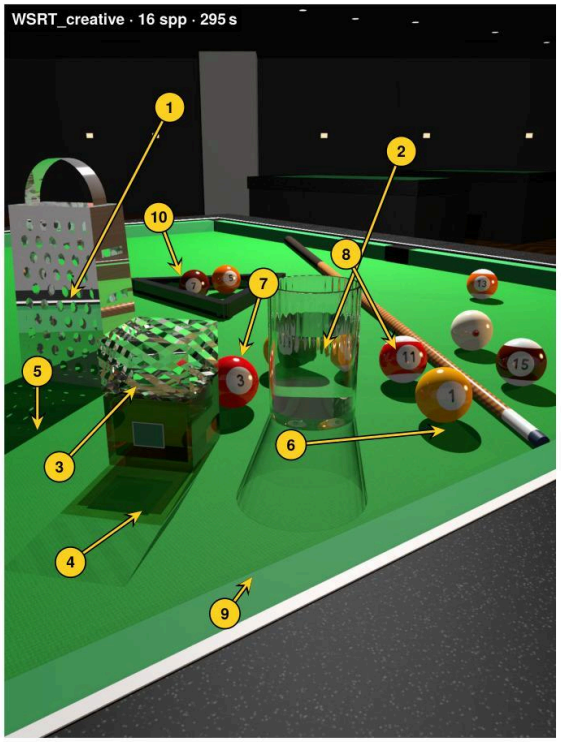
2.8 WSRT_creative

My creative scene recreates a photo I took in the Teviot Row House's pool hall (Figure 7a). Each object in it tests something: the grater test reflection, the water glass tests refraction, the perfume bottle tests coloured shadows, the balls test textured spheres and highlights. I left out the people, TV and windows in the background, and the caustic feature of the glass, which a WSRT can't produce. To make the render line up with the photo, the pbrt camera was fitted to the eight visible balls: their sizes and positions give the field of view (46.2 °), height and tilt, and the predicted ball sizes then match the photo to the highest degree possible. All meshes and textures are made by small Python scripts generated by Claude, so the scene is locally regeneratable. One bug appeared here: the whole table sat 4.18 units too close at first because of one wrong coordinate in the generator, and I only caught it when an annotation arrow missed the rail. The final render (Figure 8) takes 295 s.



(a) My photo. **(b)** WSRT_creative. **(c)** DRT_creative.

Figure 7: The photo that inspired both creative scenes, and the two renders, which share the same camera and geometry file.



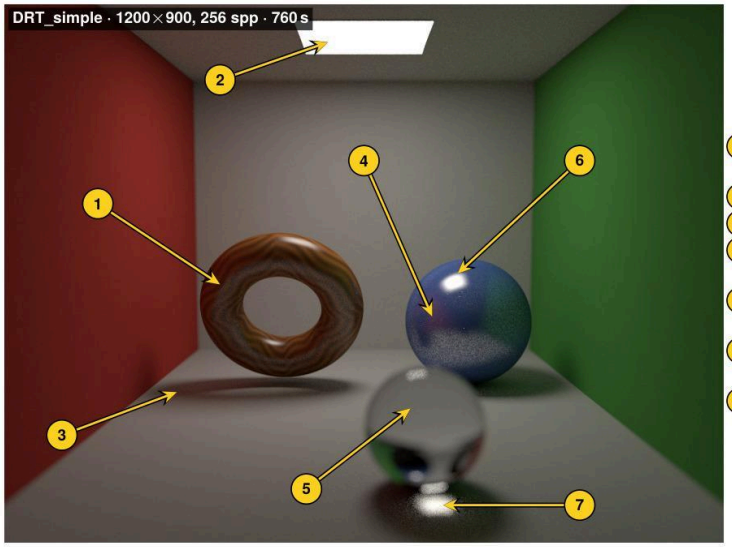
1. grater ($K_r = 0.75$): mirror reflection of the felt, cue and bottle
2. water glass: refraction, the ball behind it magnified and flipped
3. crystal cap ($\eta = 1.55$): refraction through tilted facets
4. filtered, two-tone shadow of the bottle
5. light through the grater's holes (alpha cut-outs)
6. two hard shadows per ball, key and fill
7. Blinn–Phong highlights from the white key and warm fill
8. UV-textured spheres (numbers)
9. textured triangles: felt weave and speckled rail
10. rack: one bar instanced three times

9 823 primitives in 84 objects; 2 point lights; 30.7 M camera, 74.6 M secondary and 136.5 M shadow rays; no warnings.

Figure 8: WSRT_creative annotated. Every WSRT feature appears at least once. Close-ups were already analysed in Figures 4 to 7.

3 Distributed ray tracer (DRT)

The DRT (Cook et al. 1984) keeps the Whitted structure but replaces each single, perfect ray with random samples: many samples per pixel, random points on the lens (depth of field), shadow rays to random points on area lights (soft shadows), and bounce rays in random directions over the hemisphere (glossy and indirect light). At every hit the lights are sampled directly, and a bounce ray that happens to hit a light doesn't count it again, so no light is counted twice. DRT_simple (Fig. 9) is a closed box lit by one ceiling light, so that light bounces around and the coloured walls bleed onto the floor.



1. OBJ torus (9 216 triangles) with smooth normals and a wood texture (§§ 3.1 and 3.2)
2. area light: an emissive rectangle (§ 3.3)
3. soft shadow with a penumbra (§ 3.3)
4. Phong BRDF: glossy reflection of the red wall (§ 3.4)
5. thin lens focused at 8: the near glass sphere is blurred (§ 3.5)
6. Reinhard: the highlight rolls off instead of clipping (§ 3.6)
7. caustic speckles under the glass (§ 3.4)

Figure 9: DRT_simple: every DRT feature in one image. Settings: maxdepth 3, 4 shadow rays per sample, lens radius 0.1, focal distance = 8, Reinhard on luminance with white point 1.6. No warnings occurred.

3.1 Triangle meshes

What I did and why: pbrt only reads PLY, so I added an OBJ loader (Shape "objmesh"). It reads positions, UVs, normals and faces in all of OBJ's index styles and splits polygons into triangles. Because OBJ indexes positions, UVs and normals separately, every distinct combination becomes its own vertex, which keeps seams correct (one position with two UVs). At a hit, the triangle's coordinates combine the three vertex

normals into a smooth shading normal. I also keep the flat geometric normal, for ray offsets and to reject light coming from below the true surface.

How I tested it: Figure 10a loads the same 576-triangle torus without and with normals: with them the highlight flows smoothly across triangle edges while the outline stays exactly the same. The log reports each mesh's triangle count and whether normals and UVs were found, which I then tried to compare with the scripts that made the meshes.

Challenges: Increasing speed rather than correctness: the 9 216-triangle torus only became practical to do in terms of speed with the per-object bounding boxes method from the feature 2.7.

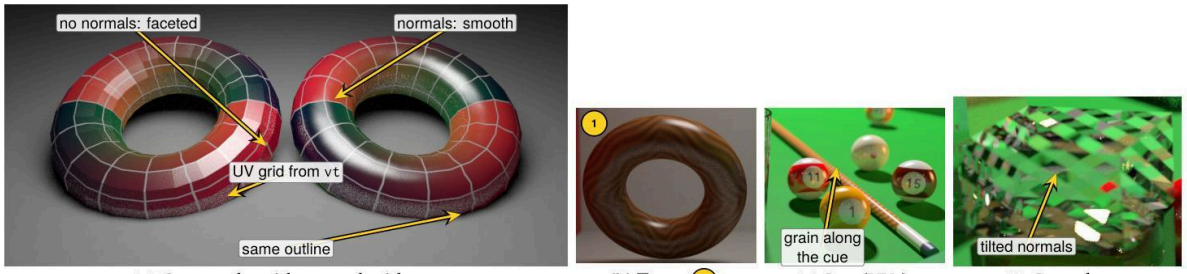
Observation. Normals alone sometimes fake details. For example, the crystal cap is a plain box whose normals tilt 18° in bands, and it sparkles like cut glass (Figure 11d).

3.2 Textures on meshes

What I did and why: Nothing new was needed: a mesh's (u,v) at a hit is the combination of its vertices' vt values, exactly like the normals, and goes into the same texture code as in the WSRT.

How I tested it: The UV grid wraps twice around the coarse toruses (uscale 2), the wood grain runs around the torus tube, and on the cue it runs along the shaft (Figure 11b, c).

Observation: I didn't observe anything unusual in this feature that is worth pointing out in my opinion.



(a) One mesh, without and with vn. **(b)** Torus 1. **(c)** Cue (UVs). **(d)** Crystal cap.

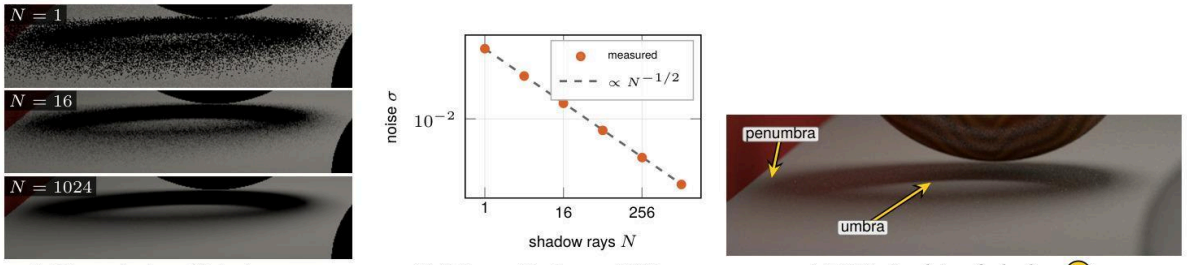
Figure 10: Triangle meshes and textures on meshes. (a) Vertex normals change the shading, not the shape: the coarse torus looks round but its outline stays visibly polygonal. (b, c) Textures follow the interpolated UVs. (d) Bands of tilted normals make a simple box look like cut crystal.

3.3 Area lights and soft shadows

What I did and why: Any shape can glow with AreaLightSource "diffusy3e" "rgb L" [..]. At each shading point the DRT sends N shadow rays tao uniformly random points on the light and averages their contributions ($f L_e \cos\theta \cos\theta' A/d^2$ for each ray that gets through). In the edge of the shadow, only some of the rays get through, which is exactly what makes the shadow soft around the edges compared to WSRT. N is set for the whole scene with "integer lightsamples" or per light with "integer samples". I sample points on the light directly because random directions would rarely hit a small light and the render would be more noisy.

How I tested it: To be able toisolate the light sampling, I rendered the torus shadow with a pinhole camera, one sample per pixel and direct light only, so the point chosen on the light is the only random choice (Fig. 11a). The noise falls as $N^{-0.501}$ over three orders of magnitude, matching the Monte Carlo $N^{-1/2}$ law (Figure 11b), and pbrt-v4 test 6 (a sphere under a rectangular light) agrees within 2.5 %. In the full scene, 16 times more shadow rays only cut the error in the shadow by 31 %, because lens and bounce noise remain, so I think a few shadow rays per sample plus more samples per pixel is the better tradeoff.

Challenges: The pool hall had 13 small lamps. As real lights the eight ceiling spots alone took half of all shadow rays for under 0.1 % of the light, and when I stopped sampling them, random bounce rays still hit them now and then and left white speckles on the walls. They are now visible-only emitters: you see them directly and in reflections, but they don't light the scene.



(a) Torus shadow, N shadow rays. **(b)** Noise vs N: slope -0.501. **(c)** DRT_simple's soft shadow (3).

Figure 11: Area lights and soft shadows. (a) With one shadow ray per pixel the edge of the shadow is pure noise, whereas with 1024 it is smooth. (b) The noise follows the $N^{-1/2}$ law of Monte Carlo integration. (c) In the final render the dark inner shadow fades into a wide edge of shadow, unlike the hard shadows of the WSRT.

3.4 Phong BRDF and energy conservation

What I did and why: The DRT uses the energy-conserving Phong BRDF (Lafortune and Willems 1994), multiplied by $\cos\theta_i$ (the cosine-weighted diffuse term):

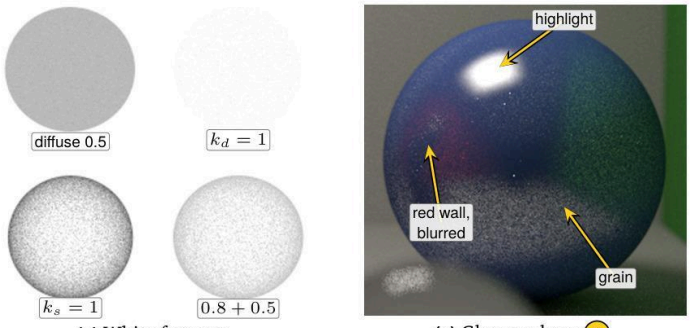
$$f(\boldsymbol{\omega}_o, \boldsymbol{\omega}_i) = \frac{k_d}{\pi} + k_s \frac{n+2}{2\pi} \max(0, \mathbf{r} \cdot \boldsymbol{\omega}_i)^n, \qquad \mathbf{r} = \text{mirror direction of } \boldsymbol{\omega}_o.$$

The constants are what make it conserve energy: k_d/π reflects exactly k_d of the incoming light, and $(n+2)/2\pi$ makes the lobe reflect exactly k_s head-on. So energy conservation comes down to $k_d + k_s + k_r \le 1$ per colour channel. If a scene breaks that, the renderer limits k_d andgives a warning. I chose to cut the diffuse term because K_s and K_r are usually tuned on purpose to adjust the look of highlights.

How I tested it: The white furnace test (Figure 12a): spheres in a uniform white environment. A material that conserves energy can never look brighter than its surroundings. The diffuse sphere gives 0.4999, the sphere wit halbedo=1 disappears into the background (0.9989), and binned by viewing angle every sphere stays within 0.02 of the numerically integrated albedo and never above 1, including the one that asked for $k_d + k_s = 1.3$.

Problems: In the first furnace test the spheres were too close together and hid part of the environment from each other, so the albedo-1 sphere read 0.94 at its rim. But then I spaced them out and it fixed it. In the creative scene, reusing the WSRT's specular values made the rail look like plastic, because the lobe's peak is much higher and its K_s went down to 0.02.

Observation. Glossy surfaces stay grainy (Figure 12c): bounce directions are uniformly distributed over the hemisphere so only few of them land in a narrow lobe. This is something to fix in later modules hopefully.



(a) White furnace. **(c)** Glossy sphere (4).

Figure 12: Phong BRDF. (a) In a uniform white environment no sphere is brighter than its background, and each matches the albedo obtained by integrating its BRDF numerically. (c) The specular lobe gives a highlight and a blurred reflection of the red wall, the grain comes from sampling the lobe uniformly.

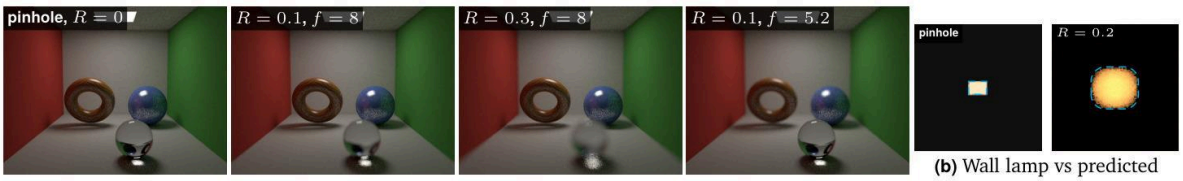
3.5 Defocus blur and depth of field

What I did and why: The camera follows pbrt's thin-lens model. Each ray starts at a random point on a lens of radius R and is aimed at the point where the pinhole ray crosses the focal plane at distance f. All rays through one pixel therefore meet again on that plane, so it stays sharp, while a point at depth z is smeared over a circle of confusion of radius $R|z - f|/z$. I set both of them in the scene with the following variables ("float lensradius", "float focaldistance").

How I tested it: (Figure. 13a) changes one setting at a time: a bigger *R* blurs more, and moving *f* from 8 to 5.2 (the glass sphere's depth) swaps which object is sharp. For a number, I compared the wall lamps in WSRT_creative (pinhole) and DRT_creative (*R* = 0.2, focused on the water glass at 23.4) from the same camera: the extra spread of each lamp gives the blur radius, and all four lamps match the predicted 11.4 pixels within 1.6 % (Figure 13b).

Problems: No problems in the camera itself. The only problem is that out of focus areas need many more samples before they actually look smooth.

Observation: I didn't observe anything else of importance in this feature.



(a) Lens radius R and focal distance f, one at a time.

(b) Wall lamp vs predicted blur.

Figure 13: Depth of field. (a) A larger lens blurs everything off the focal plane more. Focusing at 5.2 makes the near glass sphere sharp and blurs the torus. (b) A wall lamp in the creative scene: the dashed outline is the lamp widened by the predicted circle of confusion (11.4 pixels), and the measured blur matches it within 1.6 %.

3.6 Reinhard tone mapping

What I did and why: The DRT outputs HDR values (the lamps in DRT_creative reach 649), which have to be squeezed into the 0-1 range of a screen. After an optional exposure I apply Reinhard's operator with a white point L_w, which is the the brightness that maps to pure white:

$$L_d = \frac{L\,(1 + L/L_w^2)}{1 + L}, \qquad L = 0.2126R + 0.7152G + 0.0722B,$$

then convert to sRGB, while the linear PFM is saved untouched. My documented choice is luminance mode: the curve is applied to the luminance and the colour is scaled by L_d/L, which keeps every pixel's hue and saturation. If it is applied per channel, the brightest channel is gonna be compressed the most, which turns the bright green felt (pool table cloth) pale.

How I tested it: Figure 14b shows the same HDR data four ways, recomputed from the PFM with the renderer's formula (the luminance version matches the rendered PNG exactly). Clamping leaves 5.8 % of pixels pure white and flattens the felt. Reinhard on luminance keeps both the gradient and the colour.

Problems. Letting the white point default to the brightest pixel spent the whole range on the lamps and made the scene dark, so I set it explicitly, just above the brightest surface whose detail I wanted to keep (4 for the felt, 1.6 in DRT_simple).



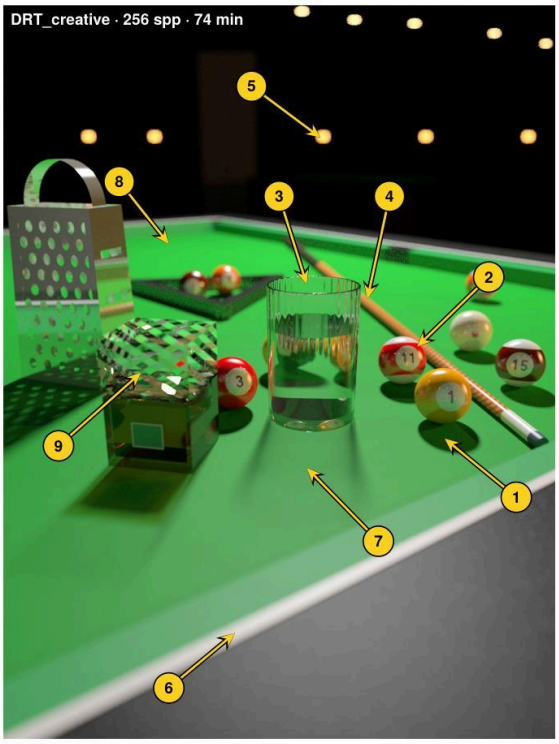
(a) The operator.

(b) DRT_creative tone-mapped four ways (top: lit felt; bottom: near rail).

Figure 14: Reinhard tone mapping. Clamping (1) flattens the brightest felt into neon green and burns out the rail's white stripe. Luminance mode (2) keeps the gradient and the green. Per channel (3) washes the felt out. Using the brightest pixel as white point (4) wastes the range on the lamps and kind of darkens the scene.

3.7 DRT_creative

DRT_creative uses exactly the same camera and geometry as WSRT_creative, with normalised Phong materials, a key light with the same total output as the WSRT's point light, and a lens focused on the water glass (256 spp, 74 minutes render. Figure 15). Figure 16 puts the two tracers side by side, and Table 1 lists the problems I encountered while rendering it. Three of the fixes are options I added (transparentshadows, minweight, caustics): they are turned off by default and not used in DRT_simple, which stays a plain DRT.



1. soft shadows from the disc key light
2. Phong BRDF: glossy highlight and clear coat
3. smooth OBJ meshes: the glass and the water
4. texture on mesh UVs: the cue's wood
5. depth of field: round bokeh of the wall lamps
6. depth of field: the near rail is blurred
7. filtered shadow of the glass, with soft edges
8. Reinhard: the brightest felt keeps its gradient
9. faceted crystal cap (tilted mesh normals)

9 824 primitives; a disc key light plus 13 visible-only lamps; 0.49 G camera, 1.75 G bounce and 13.4 G shadow rays; no warnings.

Figure 15: DRT_creative, annotated. Close-ups are analysed Figures 11, 13 to 15 and 17.

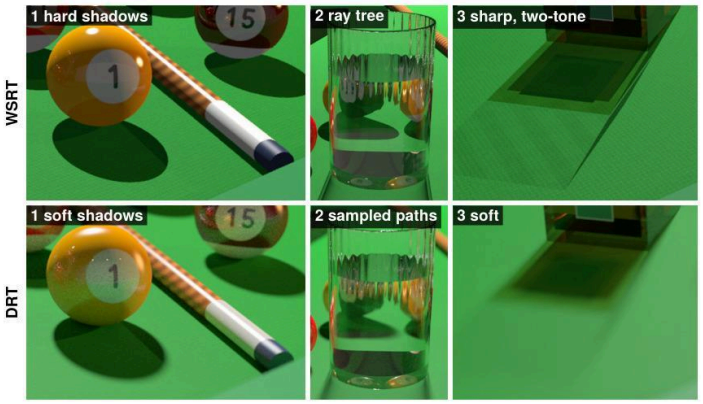


Figure 16: WSRT against DRT on the same geometry and camera. (1) A point light against a disc light with the same output: the dark core shrinks and the shadows at the edge appears to be widening out further from the ball. (2) The glass looks more or less similar, because both follow the same mirror and refraction paths. (3) The bottle's shadow gains soft edges.

Table 1: Problems I ran into while rendering DRT_creative, and how each was fixed.

What I saw	Cause	Fix	Cost
water and bottle black inside	the view through the water crosses 6 surfaces, but maxdepth was 6	maxdepth 12, and minweight so dark paths still stop early	none visible
glass shadow black with white speckles	light through the glass was only found by lucky random bounces	transparentshadows, as in the WSRT	smooth shadow, about 4.5 % too dark (no focusing)
bright dots on the felt	light reaching the small key light via the chrome grater (caustic paths)	make "bool caustics" false	1.2 % darker overall
speckles on the dark walls	8 small ceiling spots took half the shadow rays	Implement visible-only lamps (in 3.3)	3.4× faster

4 PBRT scene compatibility

Parsing: The parser has three layers. A tokenizer records the line and column of every token. A dispatcher keeps pbrt's graphics state, and typed parameter lists check each parameter's type and number of values, convert spectra to RGB, and warn about parameters that nothing used, which catches typos. Splitting it up this way allowed every error to be reported exactly where it happens.

What is supports: All transform directives, the perspective camera, spheres, discs, triangle, PLY and bilinear meshes, point, spot, distant, infinite and area lights, the diffuse, dielectric, conductor and interface materials, and image and checkerboard textures. pbrt has no Whitted integrator, no Blinn-Phong and no infinite plane, so I added them in pbrt's own syntax: Integrator "whitted"/"distributed", Material "blinnphong"/"phong"/"mirror", Shape "plane"/"objmesh" and the tone-mapping settings on Film. Rough materials and coateddiffuse are approximated, while media, environment maps, normal maps and PNG/EXR textures are not supported. Each case produces a message in the log.

Error handling: I tested 18 deliberately broken scenes generated by AI. All 18 give a clear message with file:line:column and none of them crashes (Table 2). Recoverable problems are skipped with a message, and only a fully broken token stream stops parsing (exit code 1).

Table 2: Examples from the 18 broken scenes.

Logged message	Result
13.pbrt:11:28: syntax error: unterminated string (missing closing "") parsing aborted	exit 1
10.pbrt:12:1: Rotate expects 4 numeric arguments but got 0 (found "ninety" where a number was expected)	rendered
12.pbrt:11:1: sphere radius must be > 0 (got -1.000000) - shape ignored	rendered
17.pbrt:11:1: trianglemesh: vertex index 7 out of range [0,2] - shape ignored	rendered
15.pbrt:11:1: Include: cannot open file 'missing_include.pbrt' - ignored	rendered

5 Profiling: baseline timings

How I measured: main.cpp times each stage (parsing, rendering, writing), and every thread counts its rays and ray tests. All of it goes into the timing logs' profile section. I took the final timings on mains power with caffeinate (a MacOS commando stop the Mac from sleeping), because an earlier log claimed 4071 s because the laptop had gone to sleep halfway through a render.

Table 3: Baseline timings of the four required images (Module 1 code, 15 threads).

Image	resolution, spp	primitives	parse	render	total rays	tests per ray	Mrays/s
WSRT_simple	1200×900, 4	8	0.009 s	0.11 s	15.6 M	1.7	145.3
WSRT_creative	1200×1600, 16	9 823	0.099 s	295 s	242 M	2 790	0.82
DRT_simple	1200×900, 256	9 230	0.015 s	760 s	3.41 G	633	4.49
DRT_creative	1200×1600, 256	9 824	0.136 s	4 455 s	15.7 G	664	3.51

What the numbers show: Parsing takes at most 0.14 s, rendering is over 99.9 % of the time. Speed depends on how many primitives each ray has to test, not on the number of rays. WSRT_simple tests 1.7 per ray and manages 145 M rays per second, while WSRT_creative tests about 2800 per ray, mostly because the 8064-triangle glass sits in the middle of the frame, and manages only 0.8 M. The DRT is dominated by shadow rays (86 % of the rays in DRT_creative), which are cheaper because they can stop at the first thing they hit. These numbers are important to see the inefficiencies in Module 1 and see the room for improvement in later Modules.

6 Use of generative AI

Tools and how much it did: Majority of the code (the C++ renderer, the scenes, and the scripts) was produced by **Claude Code** (model Claude Opus 5.5) in VS Code. I did a really low amount of manual coding. Claude helped me greatly to edit, compile, run and check the renderer itself. Claude most importantly helped me understand the concepts and saved me from generating each pbrt scenes by hand for creative images, it was also helpful in developing pbrt test scenes to visually test the accuracy of the features developed. I was mostly responsible for giving instructions over which features to implement, providing the formulas and conceptual requirements to implement the features I learned in lectures and redesigning/reprompting to create better scenes to reflect features for simple and creative scenes.

7 Summary

Both ray tracers implement every Module 1 feature, read pbrt-format scenes with clear error messages, and log their timings. Each feature is shown in an annotated render and checked against theoretical formulas/numbers. I am overall really content with how my simple and creative images look. I think they reflect the features implemented successfully. The main limitations are that there is no BVH yet (scenes with meshes take minutes), glossy surfaces are grainy because bounces are sampled uniformly, neither tracer produces real caustics, and textures have no mip-mapping. The next modules will hopefully tackle these problems to reach a more accurate and efficient renderer.